

Computational Biomathematics: Spatial Models of Biological Systems with AI-Enhanced Coding

Prof. Dr. Dr. h.c. Florian Rupp
Western Caspian University

Many biological systems can be understood as local interaction rules on space. The complexity we observe in nature often emerges from very simple rules applied repeatedly. In this lab, we study three biological systems using lattice-based (grid-based) models:

1. Vegetation competition on a lattice
2. Coral reef/ sessile species competition
3. Bacterial colony growth with diffusion-like spreading

A central goal of this lab is not only to simulate biological systems, but also to learn how to use **AI responsibly and effectively** for scientific programming. In particular, we will practice the following workflow:

Biological idea → **mathematical/computational structure** → **program architecture** →
AI prompt design → **code review and refinement**

After completing this lab, you should be able to:

- explain how local biological rules can generate global patterns,
- implement simple lattice-based biological simulations in Python,
- design a modular program architecture,
- formulate clear prompts for AI-assisted coding,
- critically evaluate AI-generated code.

General Setup

We represent space as an $N \times N$ lattice (grid). Each cell contains a biological state, for example

- empty,
- tree,
- algae, or
- grass,
- coral,
- bacteria.

We use Python together with:

```
import numpy as np
import matplotlib.pyplot as plt
```

Why Prompt Design Matters

When using AI for coding, it is tempting to immediately ask: “Write me Python code for this model.” However, in scientific programming this is often a poor strategy. Before asking AI to generate code, you should first think about:

- What biological states exist?
- What are the local update rules?
- What should be visualized?
- Which parts of the program should be easy to replace later?
- Which files or functions are needed?

Important Note: AI can generate code quickly, but scientific quality depends on whether *you* provide a precise model description and a clear software structure.

Recommended Program Architecture

For all three models, use a modular structure such as:

- `main.py` — runs the simulation
- `initialization.py` — creates the initial grid
- `dynamics.py` — contains the update rules
- `visualization.py` — plotting and animation
- `parameters.py` — parameters such as N , T , probabilities

Student Task

Before writing any prompt, sketch the architecture of your program on paper. For each file, write down:

- its purpose,
- the functions it should contain,
- which inputs and outputs these functions should have.

Part 1: A General Grid Framework

A simple starting point

```
def initialize_grid(N, states):  
    return np.random.choice(states, size=(N, N))
```

```
def plot_grid(grid, title=""):
    plt.imshow(grid, cmap='viridis')
    plt.title(title)
    plt.colorbar()
    plt.show()
```

Neighborhood function

We often use the 8-neighborhood (Moore neighborhood).

```
def get_neighbors(grid, i, j):
    N = grid.shape[0]
    neighbors = []
    for di in [-1, 0, 1]:
        for dj in [-1, 0, 1]:
            if di == 0 and dj == 0:
                continue
            ni = (i + di) % N
            nj = (j + dj) % N
            neighbors.append(grid[ni, nj])
    return neighbors
```

Student Task

- Explain in words what the function `get_neighbors` does.
- Why might periodic boundary conditions

$$(i + di) \% N, \quad (j + dj) \% N$$

be useful in a simulation?

Part 2: Vegetation Competition on a Lattice

Biological Idea

We consider three states: 0 = empty, 1 = grass, and 2 = tree. A possible biological interpretation is as follows:

- Empty sites may be colonized by nearby grass.
- Grass may develop into trees if trees are sufficiently present nearby.
- Trees may die due to disturbance.

Example Update Rule

```
def update_vegetation(grid):
    new_grid = grid.copy()
    N = grid.shape[0]

    for i in range(N):
        for j in range(N):
            neighbors = get_neighbors(grid, i, j)

            if grid[i, j] == 0:
                if 1 in neighbors and np.random.rand() < 0.3:
                    new_grid[i, j] = 1

            elif grid[i, j] == 1:
                if neighbors.count(2) > 3:
                    new_grid[i, j] = 2

            elif grid[i, j] == 2:
                if np.random.rand() < 0.05:
                    new_grid[i, j] = 0

    return new_grid
```

Step 1: Think about the architecture

Student Task

Before asking AI for code, answer the following:

1. What is the biological state space?
2. Which parameters should be adjustable?
3. Which file should contain the vegetation update rule?
4. Which file should contain the initialization?
5. What should be visualized?
6. How could this model later be extended?

Student Task

Write a short architecture plan. For example:

- `parameters.py`: lattice size, simulation time, colonization probability, disturbance probability
- `initialization.py`: create an initial random vegetation pattern
- `dynamics.py`: update one time step
- `visualization.py`: plot lattice and counts over time
- `main.py`: run the simulation

Step 2: Design a good AI prompt

AI Prompt Design Task

Write a prompt for an AI system that asks it to generate a modular Python program for the vegetation competition model. Your prompt should contain:

- the biological states,
- the update rules,
- the required files,
- the desired visualizations,
- the requirement that the code remain easy to extend.

AI Prompt Design Task

Compare the following two prompts:

Weak prompt:

"Write Python code for vegetation on a grid."

Strong prompt:

A detailed prompt specifying states, update rules, file structure, adjustable parameters, and visualizations.

Explain why the second prompt is much more likely to produce useful scientific code.

Step 3: Review the AI-generated code

Student Task

After receiving code from AI, check the following:

- Are the biological states represented correctly?
- Are the update rules exactly those requested?
- Is the code modular?
- Are initialization, dynamics, and visualization separated?
- Can parameters be changed easily?
- Are comments and docstrings understandable?

Simulation Task

Simulate the model for $T = 50$ time steps.

Student Task

Investigate the effect of changing:

- the initial proportion of trees,
- the colonization probability of grass,
- the disturbance probability of trees.

Describe how the spatial pattern changes.

Part 3: Coral Reef/ Sessile Species Competition

Biological Idea

We consider: 0 = empty, 1 = coral, and 2 = algae. A possible biological interpretation is as follows:

- Coral grows slowly and may be damaged by bleaching.
- Algae spreads quickly into empty sites.
- Coral and algae compete for space.

Example Update Rule

```
def update_coral(grid):
    new_grid = grid.copy()
    N = grid.shape[0]
```

```
for i in range(N):
    for j in range(N):
        neighbors = get_neighbors(grid, i, j)

        if grid[i, j] == 0:
            if 2 in neighbors and np.random.rand() < 0.5:
                new_grid[i, j] = 2
            elif 1 in neighbors and np.random.rand() < 0.2:
                new_grid[i, j] = 1

        elif grid[i, j] == 1:
            if np.random.rand() < 0.02:
                new_grid[i, j] = 0

        elif grid[i, j] == 2:
            if neighbors.count(1) > 4:
                new_grid[i, j] = 1

return new_grid
```

Step 1: Think first, prompt later

Student Task

Before designing your AI prompt, answer:

1. Which species spreads faster?
2. Which process represents bleaching?
3. Which quantities should be plotted over time?
4. Which parts of the code should remain interchangeable?

Student Task

Write down a proposed modular architecture for this coral reef model. In particular, decide:

- how to store model parameters,
- how to initialize the reef,
- how to count the numbers of coral, algae, and empty cells,
- how to visualize the lattice.

Step 2: Prompt engineering task

AI Prompt Design Task

Write a prompt for an AI system asking it to generate a modular Python simulation for coral–algae competition on a lattice. Your prompt should explicitly ask for:

- separate files,
- clear comments,
- adjustable bleaching probability,
- a lattice plot,
- a time-series plot of state counts.

AI Prompt Design Task

Add one sentence to your prompt that encourages future extensibility. For example, ask that the code be easy to modify later for:

- nutrient effects,
- climate stress,
- additional sessile species.

Step 3: Scientific review of the result

Student Task

Suppose the AI-generated code runs without errors. Why is that *not* enough to conclude that the code is scientifically correct? List at least three checks you should still perform.

Simulation Task

Run simulations for different bleaching probabilities.

Student Task

Explore:

- low bleaching probability,
- medium bleaching probability,
- high bleaching probability.

What happens to long-term coral survival?

Part 4: Bacterial Colony Growth with Diffusion-Like Spreading

Biological Idea

We consider: 0 = empty and 1 = bacteria. A possible biological interpretation is as follows:

- bacteria reproduce into neighboring empty sites,
- growth is local,
- overcrowding may reduce survival,
- the colony spreads outward from an initial seed.

Example Update Rule

```
def update_bacteria(grid):
    new_grid = grid.copy()
    N = grid.shape[0]

    for i in range(N):
        for j in range(N):
            neighbors = get_neighbors(grid, i, j)

            if grid[i, j] == 0:
                if 1 in neighbors and np.random.rand() < 0.4:
                    new_grid[i, j] = 1

            elif grid[i, j] == 1:
                if neighbors.count(1) > 6:
                    if np.random.rand() < 0.2:
                        new_grid[i, j] = 0

    return new_grid
```

Initial Condition

A simple initial condition is one bacterium in the center:

```
grid = np.zeros((N, N))
grid[N//2, N//2] = 1
```

Step 1: Architecture before prompting

Student Task

Think about the structure of the program. Which of the following belong in:

- `initialization.py`
- `dynamics.py`
- `visualization.py`
- `main.py`

Items to place:

- create the initial single-bacterium colony,
- implement spreading and overcrowding,
- plot the lattice,
- plot the number of occupied cells over time,
- run the simulation loop.

Student Task

Why is it useful to separate the simulation loop from the update rule? Why is it useful to separate plotting from model dynamics?

Step 2: Prompt design task

AI Prompt Design Task

Write a prompt for an AI system that asks for a modular Python simulation of bacterial colony growth on an $N \times N$ grid. Your prompt should specify:

- the states,
- the local update rules,
- the initial condition,
- the need for a lattice visualization,
- the need for a graph of occupied cells over time,
- the desired file structure.

AI Prompt Design Task

Now improve your prompt by adding one or two sentences that ask the AI to make the code easy to extend to:

- nutrient-dependent growth,
- diffusion of a chemical signal,
- multiple bacterial strains.

Step 3: Review and refine

Student Task

After the AI generates code, inspect it and answer:

1. Does the implementation match the biological description?
2. Does the colony start in the center?
3. Are the update rules applied synchronously?
4. Are the visualizations correct and informative?
5. Which part of the code would you modify first if you wanted to add nutrients?

Simulation Task

Run the model for different spreading probabilities.

Student Task

Observe:

- how fast the colony expands,
- whether dense regions form,
- whether overcrowding changes the colony shape.

Compare your observations with biological intuition.

Part 5: General Simulation Loop

A general simulation loop may look like this:

```
def simulate(grid, update_func, T):  
    history = []  
    for t in range(T):  
        history.append(grid.copy())  
        grid = update_func(grid)
```

```
return history
```

Student Task

Explain why the command `grid.copy()` is important here. What could go wrong if we only wrote `history.append(grid)`?

Part 6: How to Judge an AI Prompt

A strong prompt for scientific programming usually contains:

- the biological model,
- the state space,
- the update rules,
- the initial condition,
- the desired outputs,
- the program architecture,
- a requirement for modularity and readability.

Student Task

Take one of your own prompts from the earlier tasks and evaluate it according to the following checklist:

1. Does it describe the biology clearly?
2. Does it specify the architecture?
3. Does it say what should be visualized?
4. Does it require modular code?
5. Does it make future extensions possible?

Part 7: Reflection on AI-Assisted Coding

Student Task

Write a short reflection (about half a page) on the following questions:

- What part of the programming task was most difficult to describe precisely?
- How did thinking about the architecture improve your prompt?
- Did the AI-generated code fully match your intended biological model?
- What errors or weaknesses did you detect?
- In what way was AI useful, and in what way was human judgment still essential?

Discussion Questions

1. How do local rules influence global patterns?
2. Which of the three systems is most sensitive to parameter changes?
3. Which model is biologically the most realistic? Which is the simplest?
4. What aspects of biology are still missing from these models?
5. How can modular coding make scientific models easier to improve?
6. Why is prompt design an important scientific skill when using AI?

Advanced Extensions

- Add more species.
- Introduce spatial heterogeneity.
- Include nutrients or environmental forcing.
- Add diffusion of chemical substances.
- Compare lattice-based models with differential equation models.

Key Takeaway

In computational biology, AI is most useful when we first think carefully about the biological assumptions and the software architecture. A strong prompt begins with a strong understanding of the model.

Assessment and Grading Criteria

This lab evaluates not only your final simulation results, but also your ability to:

- translate biological ideas into computational models,
- design a clear program architecture,
- formulate high-quality AI prompts,
- critically evaluate AI-generated code,
- reflect on the role of AI in scientific work.

Grading Components

Component	Weight
Model understanding (biology + rules)	20%
Program architecture design	20%
AI prompt quality	25%
Code correctness and modularity	20%
Reflection on AI usage	15%

Rubric for AI Prompt Design

Your AI prompts will be assessed according to the following criteria.

1. Clarity of Biological Model

- **Excellent (5):** States, rules, and assumptions are precise, unambiguous, and complete.
- **Good (4):** Mostly clear, minor ambiguities.
- **Satisfactory (3):** Basic idea present but incomplete.
- **Weak (2):** Important aspects missing or unclear.
- **Poor (1):** Model not clearly described.

2. Specification of Program Architecture

- **Excellent (5):** Clear modular structure (files, functions, responsibilities).
- **Good (4):** Structure mostly specified.
- **Satisfactory (3):** Some structure but incomplete.
- **Weak (2):** Minimal structure indicated.
- **Poor (1):** No structure specified.

3. Precision of Instructions to AI

- **Excellent (5):** Detailed, precise, and well-scoped instructions.
- **Good (4):** Clear but missing some details.
- **Satisfactory (3):** Understandable but vague.
- **Weak (2):** Ambiguous instructions.
- **Poor (1):** Very unclear or incomplete.

4. Consideration of Outputs and Visualization

- **Excellent (5):** Clearly specifies what should be visualized and how.
- **Good (4):** Visualization mentioned but not detailed.
- **Satisfactory (3):** Basic output requested.
- **Weak (2):** Output unclear.
- **Poor (1):** No output specified.

5. Modularity and Extensibility

- **Excellent (5):** Explicitly requires modular, extensible code.
- **Good (4):** Mentions modularity.
- **Satisfactory (3):** Implicit structure.
- **Weak (2):** Little concern for extensibility.
- **Poor (1):** No mention of modularity.

6. Scientific Awareness

- **Excellent (5):** Prompt reflects awareness of validation, interpretation, or limitations.
- **Good (4):** Some awareness present.
- **Satisfactory (3):** Limited reflection.
- **Weak (2):** Minimal scientific thinking.
- **Poor (1):** No scientific considerations.

Rubric for Code Evaluation

1. Correctness of Implementation

- **Excellent (5):** Fully matches biological rules.
- **Good (4):** Minor inconsistencies.
- **Satisfactory (3):** Some errors but usable.
- **Weak (2):** Major errors.
- **Poor (1):** Incorrect model.

2. Modularity and Structure

- **Excellent (5):** Clear separation of components.
- **Good (4):** Mostly modular.
- **Satisfactory (3):** Some structure.
- **Weak (2):** Poor organization.
- **Poor (1):** Monolithic code.

3. Readability and Documentation

- **Excellent (5):** Clear comments and docstrings.
- **Good (4):** Mostly readable.
- **Satisfactory (3):** Some comments.
- **Weak (2):** Hard to understand.
- **Poor (1):** No documentation.

Self-Assessment and Reflection

Student Task

Evaluate your own AI prompt using the rubric above.
For each category, assign yourself a score from 1 to 5 and briefly justify your choice.

Student Task

Reflect on the following:

1. What was the biggest improvement between your first and final prompt?
2. Which part of the model was hardest to describe precisely?
3. Did the AI-generated code behave as expected? If not, why?
4. What errors did you identify in the AI output?
5. What would you change if you had to write the prompt again?

Student Task

Meta-question:

Do you think writing a good prompt is more similar to:

- writing mathematical definitions, or
- writing computer code?

Explain your answer.

This lab is designed to emphasize that:

AI does not replace modelling or scientific reasoning — it amplifies it when used with precision.

Students should understand that:

- vague prompts lead to poor scientific code,
- clear structure leads to reusable models,
- verification is essential even when code runs without errors.

Final Takeaway

The quality of AI-generated scientific code depends directly on the quality of the prompt, and the quality of the prompt depends on the clarity of the underlying scientific thinking.